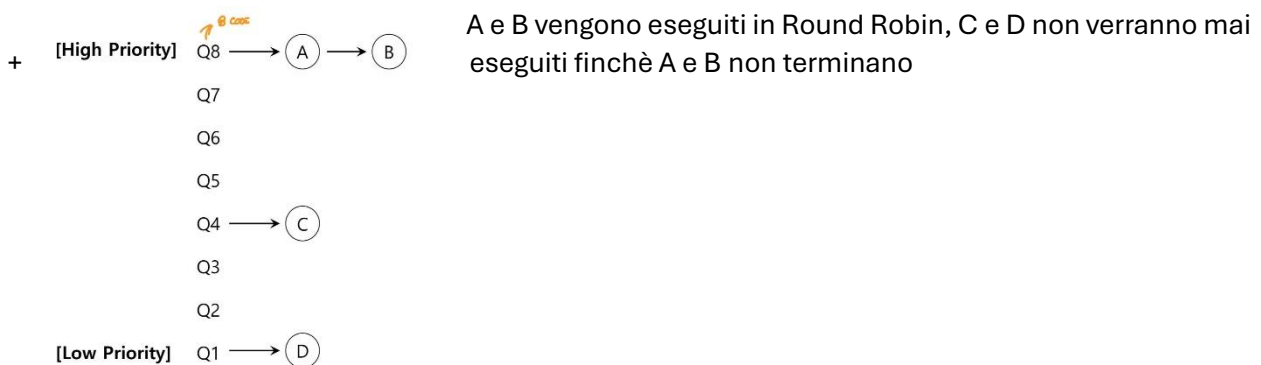


Multi-Level Feedback Queue (MLFQ)

Con le altre politiche di scheduling abbiamo notato che per migliorare il turnaround time ottenevamo un tempo di risposta pessimo mentre per migliorare il tempo di risposta ottenevamo un turnaround time più elevato. Quello che tenta di fare il MLFQ è di ottimizzare il turnaround time e minimizzare il tempo di risposta, senza sapere a priori il tempo di esecuzione del processo. Questa politica di scheduling impara a run-time le caratteristiche dei job da eseguire, in modo tale da prendere delle decisioni migliori. MLFQ ha un numero di code distinte, ogni coda ha un livello di priorità.

RULE 1: se la $PRIORITA(a) > PRIORITA(b)$ viene eseguito a

RULE 2: se la $PRIORITA(a) = PRIORITA(b)$ a e b vengono eseguiti in RR



With the rules we have just defined, A and B will be executed in a round robin fashion. Jobs C and D will never run, as long as A and B don't terminate

MLFQ cambia la priorità di un job osservandone il comportamento:

Se un job rilascia spesso la CPU per fare un'operazione di I/O gli lascia la priorità alta dato che probabilmente è un job interattivo e vogliamo che i job interattivi siano reattivi. Se un job utilizza la CPU per un lungo periodo di tempo gli riduce la priorità, questo perché si sta comportando come un job batch (es. avvio un calcolo la sera e la mattina ne vedo il risultato). Aggiungiamo delle regole al cambio di priorità:

RULE 3: Quando un job entra nel sistema gli viene attribuita la massima priorità

RULE 4a: Quando un job usa un intero quanto di tempo la sua priorità viene ridotta, questo perché potrebbe essere un job batch che richiede la cpu e non esegue operazioni di I/O

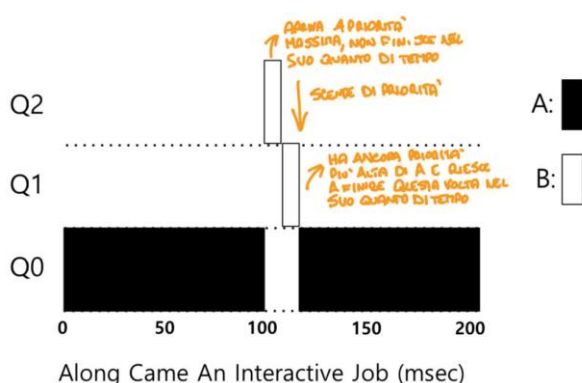
RULE 4b: Se un job rilascia la CPU prima che finisca il quanto di tempo la sua priorità rimane allo stesso livello, questo perché potrebbe essere un job interattivo

Esempio:

Il job A ha già avuto modo di dimostrare che non è un job interattivo

Il job B è un job interattivo che ha runtime 20ms

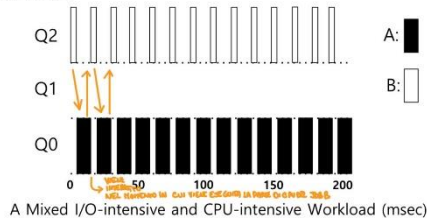
A è stato eseguito per una certa quantità di tempo, successivamente arriva B al tempo $T=100ms$



B arriva a massima priorità e non riesce a finire la sua esecuzione, gli viene scalato un livello di priorità, ha ancora priorità più alta di A e riesce a finire questa volta nel suo quanto di tempo

Comparando MLFQ con SJF (Short Job First) SJF doveva sapere a priori la lunghezza di ogni processo, con il set di regole che abbiamo messo MLFQ approssima SJF. La differenza è che MLFQ non gli serve sapere a priori la lunghezza del job, quando arriva un nuovo processo viene messo alla priorità massima, infatti se fosse un processo corto resterebbe alla massima priorità e lavorerebbe come SJF, se invece il processo fosse lungo, quest'ultimo, calerebbe di priorità.

- o **Job A:** A long-running CPU-intensive job
- o **Job B:** An **interactive job** that needs the CPU only for 1ms before performing an I/O



Quest'esempio ci fa notare delle debolezze:

-**Starvation (morire di fame):** Se ci fossero troppi job interattivi, i job a priorità più bassa potrebbero andare in starvation.

-**Ingannare lo scheduler:** Un job malevolo, potrebbe, usare il 99% del quanto di tempo e successivamente fare una richiesta I/O in modo tale da non perdere di priorità

RULE 5 (Priority Boost): Dopo un certo periodo di tempo, sposto tutti i job a priorità massima ("gli do una seconda possibilità"), in poche parole dopo un po' un job batch verrebbe messo a RR con i job interattivi.

RULE 4: (Riscrivo 4a e 4b) Il job passa a un livello di priorità inferiore se viene eseguito per il tempo del quanto di tempo. Es: Ho un job malevolo e ho quanti di tempo da 10ms, utilizza 9ms e poi fa una richiesta di I/O quando verrà poi rieseguito, avrà a disposizione solo 1ms, dopodichè scenderebbe di priorità

MLFQ solitamente attribuisce ai livelli di priorità più alti quanti di tempo minori e viceversa.

L'implementazione Solaris di MLFQ ha 60 code, e incrementa lentamente i quanti di tempo: Livello di priorità massimo: 20ms, Livello di priorità minimo qualche centinaia di ms. Il priority boost viene fatto ogni secondo.